HitPredict revisited: predicting Billboard Hot 100 hits from Spotify data

UE24CS352A Machine Learning, Mini-Project, Problem 16
Team: Vishnu Patruni (PES1UG24AM187) and Piyush G (PES1UG24AM190)

1. Problem statement

Our problem is hit song prediction. Given a song's Spotify audio features and whether its artist has charted before, the task is to predict whether the song reached the Billboard Hot 100. The reference paper is *HitPredict* by Georgieva, Suta and Burton (Stanford CS229, 2018). It reports about 75% accuracy, with logistic regression (LR) and a small neural network (NN) as the best models. We set out to reproduce its experiments, check whether its evaluation was fair, and improve on it only with changes we could justify.

2. Dataset

We used the paper's own data, which the authors published in the code archive linked from the paper. Collecting it again is not possible, because Spotify closed its audio-features API to new applications in 2024. The file has 4,040 rows. After removing 21 exact duplicates we have 4,019 songs, of which 1,595 (39.7%) were Hot 100 hits between 1990 and 2018; the rest are random Million Song Dataset tracks. Each song has the nine audio features the paper uses (danceability, energy, speechiness, acousticness, instrumentalness, liveness, valence, loudness, tempo), Spotify's key and mode, and the authors' artist score, which is 1 if the artist had an earlier Hot 100 hit. 3,206 songs also have a release month, and the paper's main experiments use only these. Two properties of the data matter later. First, 34% of dated songs are listed as released in January, because Spotify stores a year-only release date as 1 January. Second, the hit rate ranges from 26% to 59% across five-year periods, since the non-hits were sampled without regard to year.

3. Approach

We first reran every experiment exactly as the authors did: their CSV in file order, the first 2,416 rows for training and the remaining 805 for validation, their hand-picked feature subsets and unscaled features. This covers unregularised LR fitted by Newton's method, Gaussian discriminant analysis (GDA) with a shared covariance, SVMs with linear, RBF and polynomial kernels, a decision tree, the authors' NN (one hidden layer of 6 sigmoid units), two-component EM clustering, bagging, a tree-depth sweep, forward feature selection, and the seasonal and five-year-period studies.

We then re-evaluated everything with a protocol that never uses test data to make a choice. Songs are split by lead artist, so that a featured credit such as "drake feat. rihanna" counts as "drake". Three quarters of the artists (3,015 songs) are used for training, and the other quarter (1,004 songs by 534 artists) forms a test set that is scored once per model. Hyperparameters, feature subsets, the NN epoch and the final model are all chosen by 5-fold artist-grouped cross-validation (CV) on the training artists, and 95% confidence intervals come from resampling whole artists 2,000 times. The paper instead chose its feature subsets, NN epoch, tree depth and bagging setting on the validation set and reported that same validation score, and it states that "an additional test set was not needed".

On this split we tuned 13 models by grid search, from a majority-class baseline to random forest and gradient boosting. We then added four improvements one at a time, fixed before we looked at any test score. I1 also trains on the 813 songs without a release month, which the paper left out of its LR, GDA and SVM experiments even though the date is never an input. I2 takes log10 of the four skewed features, drops key, adds artist score x audio interaction terms and tunes the L2 penalty. I3 replaces LR with gradient boosting or a soft-voting ensemble, whichever has the higher CV accuracy. I4 picks the lowest decision threshold that reaches 80% precision on out-of-fold training predictions, because the paper points out that a false positive is costly for a record label.

4. Implementation

The code is a Python package with one module per experiment family: supervised models, EM, overfitting studies, feature ablation, temporal effects, split comparison, model comparison and improvements. A single script runs all of them in about 10 minutes and writes results.json, eleven PNG figures and the trained models; a separate script downloads the data and checks its SHA-256 hashes. The random seed is fixed at 42 and package versions are pinned (Python 3.12, scikit-learn 1.9.1). Nine pytest smoke tests run in a few seconds, one of which checks that the paper's LR result is reproduced exactly. A Streamlit app loads the saved models and scores real test songs, with a command-line fallback for the live demo.

5. Results

LR, GDA and the linear and RBF SVMs match the paper's results table to three decimals (Table 1, Figure 1), the polynomial SVM is 0.1 points off, and the LR confusion matrix is identical (225 true positives, 122 false negatives, 72 false positives, 386 true negatives). The RBF and polynomial SVMs matched only after we used the 2018 scikit-learn definition of gamma='scale', 1/(n_features x std(X)); later versions use the variance instead. The decision tree is 1.4 points off and the NN 1.6 points off. We could not reproduce the NN training curve: in our runs the network did better without the paper's penalty term (best validation accuracy 76.9% against 74.9% with it), which is the opposite of the paper's finding.

Reproducing the numbers exposed three scoring bugs in the authors' scripts. The SVM training accuracies of 0.550, 0.573 and 0.543 come from comparing an (m, 1) prediction array with an (m,) label array, which numpy broadcasts to an m x m matrix. The real training accuracies are 0.721, 0.642 and 0.735, so the paper's conclusion that the SVMs overfit does not hold. The same bug produces the EM accuracies of 50.1% and 46.9%. Scored correctly, EM reaches 62.6% and semi-supervised EM 68.5%, against a majority rate of 58.6%. In the bagging study the averaged votes were compared with the 0/1 label, so a song counted as correct only when every bag agreed. This explains accuracies as low as 2.7%, while a real majority vote gives 72.3% to 75.5% in every setting.

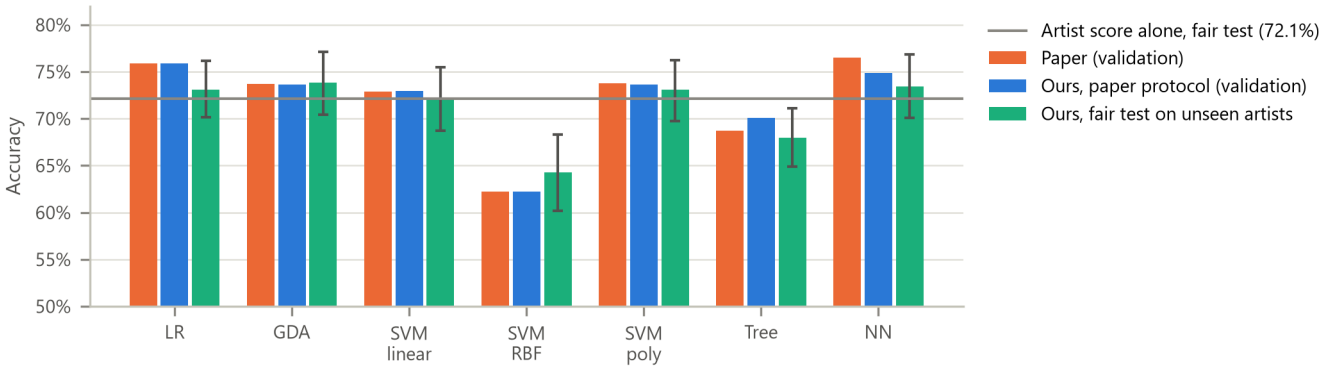


Figure 1. Validation accuracy reported by the paper, our rerun of the paper's protocol, and test accuracy of the same fixed models on unseen artists (error bars: 95% CI). The grey line is the artist score alone.

A. Supervised models (paper's Table 2)	**Paper, validation**	**Ours, paper protocol**	**Ours, fair test [95% CI]**
Logistic regression	75.9%	75.9%	73.1% [70.2, 76.2]
GDA	73.7%	73.7%	73.8% [70.4, 77.1]
SVM, linear	72.9%	72.9%	72.1% [68.7, 75.5]
SVM, RBF	62.2%	62.2%	64.2% [60.2, 68.3]
SVM, polynomial	73.8%	73.7%	73.1% [69.8, 76.3]
Decision tree	68.7%	70.1%	67.9% [64.9, 71.1]
Neural network (best epoch)	76.5%	74.9%	73.4% [70.1, 76.8]
B. Results affected by scoring bugs	**Paper**	**Ours, authors' scoring**	**Ours, corrected**
SVM linear, training accuracy	55.0%	55.0%	72.1%
SVM RBF, training accuracy	57.3%	57.3%	64.2%
SVM polynomial, training accuracy	54.3%	54.3%	73.5%
EM clustering, unsupervised	50.1%	50.0%	62.6%
EM, semi-supervised (20% labels)	46.9%	48.2%	68.5%
Bagged trees (50 bags of 200 songs)	2.7%	1.5%	75.5%
C. Fair protocol (1,004 test songs, unseen artists)	**Grouped CV acc.**	**Test acc. [95% CI]**	**Test ROC AUC**
Majority class	60.3%	60.4% [54.3, 65.7]	-
Artist score alone	73.3%	72.1% [68.7, 75.5]	0.669
B0 Paper baseline, LR on dated songs	73.9%	73.1% [70.1, 76.2]	0.815
I1 + all labelled songs	75.5%	73.1% [70.2, 76.2]	0.814
I2 + engineered features (tuned LR)	75.6%	74.3% [71.2, 77.5]	0.821
I3 + gradient boosting	76.7%	75.3% [72.2, 78.5]	0.837
Best single model: gradient boosting, 10 features	76.8%	75.9% [72.7, 78.9]	0.836

Table 1. All accuracies in percent. Section B compares each number as the authors' code computes it with the corrected value. In section C every row is scored on the same test songs; B0 to I3 are the cumulative improvement steps.

Under the fair protocol the paper's LR scores 73.1% on unseen artists, and simply predicting a hit whenever the artist had one before scores 72.1%. The audio features add only a few points. Choosing features on the validation set, as the paper did, overstated accuracy by 1.7 points on average over 20 splits, against 1.0 points with no selection. Two effects we expected did not appear. A random row split puts 56% of the paper's validation songs next to a song by the same artist in training, yet it changed accuracy by only -0.3 to +0.6 points across four models and 20 seeds. The artist score already carries what the artist's identity would. The paper's holiday effect is mostly the January artifact: out-of-fold accuracy is 78.4% for November and December songs but 71.3% for January ones.

Among the 13 models, tuned gradient boosting had the best CV accuracy (76.8%) and scored 75.9% on test; the 11 learned models lie between 68.2% and 75.9%. The improvements (Table 1, section C) raised test accuracy from 73.1% to 75.3% and ROC AUC from 0.815 to 0.837. I1 did not change test accuracy at all. It only reduced the spread of the CV estimate from 2.3 to 1.1 points, which fits a seven-feature linear model that is limited by bias rather than by the amount of data. I2 added 1.2 points, and its interaction terms are easy to read: one standard deviation more danceability raises the log-odds of a hit by 1.01 for new artists but only by 0.43 for established ones. I3 added another 1.0 point with shallow trees (depth 2, 300 rounds), which capture step effects that a linear model can only approximate: only 4.7% of songs with instrumentalness above 0.5 are hits. I4, with a threshold of 0.64, raised test precision from 69.9% to 77.5% at the cost of recall (66.1% to 46.7%). The 95% interval of the total gain, [-0.3, +4.6] points, still includes zero.

6. Conclusions

The paper's headline numbers are reproducible, but several of its conclusions are not. The SVMs do not overfit, EM is not at chance level, bagging does not collapse, and the holiday effect is largely a date artifact. On artists the model has never seen, Spotify features plus the artist's history reach about 75% accuracy, and most of that comes from the artist score alone. Our changes add 2.2 points, which is within the noise of a 1,004-song test set, so we report them as a modest gain rather than a clear win. The main limitations are the fixed feature set (no new features can be fetched from Spotify), non-hits that are random tracks rather than near-misses, and a test set too small to separate models that are less than about 3 points apart. Release year predicts the label in this data only because of how the non-hits were sampled, so we did not use it as a feature.